

Formal Verification: Hybrid Signature Scheme

Zach Kelling, Lux Industries

December 2025

Abstract

We formalize the Lux hybrid signature scheme combining Ed25519 (classical), ML-DSA-65 (lattice-based), and SLH-DSA-128s (hash-based). Full verification requires all three to pass. An attacker must break all three cryptographic assumptions to forge a signature.

1 Introduction

The hybrid scheme provides defense in depth for Lux validator identities. In normal operation, fast mode (Ed25519 + ML-DSA) balances performance and post-quantum security. Full mode adds SLH-DSA as a hash-based backup. Classical mode is a last-resort fallback.

2 Definitions

Definition 1 (HybridPK). *Hybrid public key: Ed25519 + ML-DSA + SLH-DSA components.*

Definition 2 (HybridSig). *Hybrid signature: one signature from each scheme.*

Definition 3 (VerifyMode). *Three modes: fast (Ed25519 + ML-DSA), full (all three), classical (Ed25519 only).*

3 Theorems and Axioms

Theorem 4 (full_requires_all_three). *Full verification requires all three schemes to verify: Ed25519 AND ML-DSA AND SLH-DSA.*

Proof. Decompose the nested Boolean conjunction. □

Theorem 5 (fast_requires_both). *Fast mode requires both Ed25519 and ML-DSA.*

Theorem 6 (full_implies_fast). *Full verification implies fast verification.*

Proof. Extract the first two components from full_requires_all_three. □

Theorem 7 (full_implies_classical). *Full verification implies classical (Ed25519) verification.*

4 Lean Source

```
/-
  Post-Quantum Hybrid Signature Correctness

  Models the hybrid signature scheme combining:
  - Ed25519 (classical, 32-byte keys)
  - ML-DSA-65 (FIPS 204, lattice-based, 1952-byte keys)
  - SLH-DSA-128s (FIPS 205, hash-based, 32-byte keys)

  The hybrid construction requires ALL three to verify.
  An attacker must break all three to forge - defense in depth.

  Maps to:
  - luxfi/crypto: hybrid package
  - lux/node: validator identity keys

  Properties:
  - Defense in depth: full verify all three verify
  - Classical fallback: fast mode uses ed25519 + ML-DSA only
  - Hash-based backup: SLH-DSA protects if lattices fall
-/

import Mathlib.Data.Nat.Defs
import Mathlib.Tactic

namespace Crypto.Hybrid

/-- Abstract signature verification for each scheme -/
axiom ed25519_verify : Nat → Nat → Nat → Bool
axiom mldsa_verify : Nat → Nat → Nat → Bool
axiom slhdsa_verify : Nat → Nat → Nat → Bool

/-- Hybrid public key: classical + lattice + hash-based -/
structure HybridPK where
  ed25519 : Nat
  mldsa : Nat
  slhdsa : Nat

/-- Hybrid signature: one from each scheme -/
structure HybridSig where
  ed25519 : Nat
  mldsa : Nat
  slhdsa : Nat

/-- Verification modes -/
inductive VerifyMode where
  | fast          -- ed25519 + ML-DSA (performance)
  | full          -- all three (maximum security)
  | classical     -- ed25519 only (fallback)

/-- Fast verification: ed25519 ML-DSA -/
def verify_fast (pk : HybridPK) (msg : Nat) (sig : HybridSig) : Bool :=
  ed25519_verify pk.ed25519 msg sig.ed25519 &&
```

```

mldsa_verify pk.mldsa msg sig.mldsa

/-- Full verification: ed25519 ML-DSA SLH-DSA -/
def verify_full (pk : HybridPK) (msg : Nat) (sig : HybridSig) : Bool :=
  ed25519_verify pk.ed25519 msg sig.ed25519 &&
  mldsa_verify pk.mldsa msg sig.mldsa &&
  slhdsa_verify pk.slhdsa msg sig.slhdsa

/-- Mode-selected verification -/
def verify (pk : HybridPK) (msg : Nat) (sig : HybridSig) (mode : VerifyMode) :
  Bool :=
  match mode with
  | .fast => verify_fast pk msg sig
  | .full => verify_full pk msg sig
  | .classical => ed25519_verify pk.ed25519 msg sig.ed25519

/-- DEFENSE IN DEPTH: Full verification requires all three schemes.
    An attacker must break ed25519 AND ML-DSA AND SLH-DSA. -/
theorem full_requires_all_three (pk : HybridPK) (msg : Nat) (sig : HybridSig)
  (h : verify_full pk msg sig = true) :
  ed25519_verify pk.ed25519 msg sig.ed25519 = true
  mldsa_verify pk.mldsa msg sig.mldsa = true
  slhdsa_verify pk.slhdsa msg sig.slhdsa = true := by
  simp only [verify_full, Bool.and_eq_true] at h
  obtain h12, h3 := h
  obtain h1, h2 := h12
  exact h1, h2, h3

/-- Fast mode requires both ed25519 and ML-DSA -/
theorem fast_requires_both (pk : HybridPK) (msg : Nat) (sig : HybridSig)
  (h : verify_fast pk msg sig = true) :
  ed25519_verify pk.ed25519 msg sig.ed25519 = true
  mldsa_verify pk.mldsa msg sig.mldsa = true := by
  simp only [verify_fast, Bool.and_eq_true] at h
  exact h

/-- Full implies fast: if all three verify, the fast pair certainly does -/
theorem full_implies_fast (pk : HybridPK) (msg : Nat) (sig : HybridSig)
  (h : verify_full pk msg sig = true) :
  verify_fast pk msg sig = true := by
  have h1, h2, _ := full_requires_all_three pk msg sig h
  simp [verify_fast, h1, h2]

/-- Full implies classical: if all three verify, ed25519 certainly does -/
theorem full_implies_classical (pk : HybridPK) (msg : Nat) (sig : HybridSig)
  (h : verify_full pk msg sig = true) :
  verify pk msg sig .classical = true := by
  have h1, _, _ := full_requires_all_three pk msg sig h
  simp [verify, h1]

end Crypto.Hybrid

```

5 References

Source code: <https://github.com/luxfi/proofs/blob/main/lean/Crypto/Hybrid.lean>

White papers: <https://papers.lux.network>

Related white papers:

- `lux-ethfalcon-post-quantum.tex`